

NUST Access Control System - Complete Code Explanation

Table of Contents

1. What Does This Program Do?
 2. Import Statements
 3. Function 1: Load Valid Student IDs
 4. Function 2: Load Face Database
 5. Initialize the System
 6. Main Program Loop
 7. Stage 1: Barcode Scanning
 8. Stage 2: Face Verification
 9. Exit and Cleanup
-

What Does This Program Do?

This is a **two-stage security system** for NUST campus entry:

Stage 1: Student scans their ID card barcode

Stage 2: System verifies their face matches the ID

If both match →  Access Granted

If either fails →  Access Denied

Import Statements

```
python

import cv2
from pyzbar.pyzbar import decode
from openpyxl import load_workbook
import face_recognition
import os
import time
```

What Are Imports?

Imports bring in **libraries** (pre-written code) so you don't have to write everything from scratch.

Library	What It Does
cv2	Computer vision - handles camera, images, and video
decode from pyzbar	Reads and decodes barcodes/QR codes
load_workbook from openpyxl	Opens and reads Excel files (.xlsx)
face_recognition	Detects faces and compares them
os	Works with files and folders on your computer
time	Handles timing (stopwatches, delays, etc.)

Function 1: Load Valid Student IDs

```
python
def load_cms_ids(file_path):
```

◆ What is a Function?

A **function** is a reusable block of code. Instead of writing the same code multiple times, you write it once in a function and call it whenever needed.

- `def` = "define" a new function
- `load_cms_ids` = the name we chose
- `file_path` = input parameter (the Excel file location)

```
python
wb = load_workbook(file_path)
```

Opens the Excel file and stores it in variable `wb` (short for "workbook")

```
python
sheet = wb.active
```

Gets the first sheet from the Excel file (most Excel files have multiple sheets, like tabs)

```
python
```

```
cms_ids = []
```

Creates an empty list to store all valid student IDs

What's a list? A collection that can hold multiple items:

```
python
```

```
my_list = ["apple", "banana", "cherry"]
```

```
python
```

```
for row in sheet.iter_rows(min_row=2, max_col=1, values_only=True):
```

Loop through each row in the Excel sheet

Parameter	Meaning
<code>min_row=2</code>	Start from row 2 (skip header row)
<code>max_col=1</code>	Only read column 1 (the first column)
<code>values_only=True</code>	Get just the data, not formatting

What's a loop? Code that repeats. Instead of writing 100 lines for 100 students, write 1 loop!

```
python
```

```
if row[0] is not None:
```

Check if the cell has data

- `row[0]` = First column in this row
 - `is not None` = Make sure it's not empty
 - `None` in Python = "nothing" or "no value"
-

```
python
```

```
cms_ids.append(str(row[0]))
```

Add the student ID to our list

- `str(row[0])` = Convert to text (in case it's stored as a number)
- `.append()` = Add to the end of the list

Example:

```
python
```

```
my_list = []  
my_list.append("545895") # Now list = ["545895"]  
my_list.append("557955") # Now list = ["545895", "557955"]
```

```
python
```

```
return cms_ids
```

Send the list back to whoever called this function

What's return? Functions can give back a result:

```
python
```

```
def add(a, b):  
    return a + b  
  
result = add(5, 3) # result = 8
```



Function 2: Load Face Database

```
python
```

```
def load_face_database(folder_path):
```

Creates a function that loads all face photos from a folder

```
python
```

```
face_db = {}
```

Creates an empty dictionary

What's a dictionary? Key-value pairs (like a real dictionary: word → definition)

```
python
```

```
phone_book = {  
    "Alice": "555-1234",  
    "Bob": "555-5678"  
}
```

In our case: `student_id → face_encoding`

```
python
```

```
for filename in os.listdir(folder_path):
```

Loop through every file in the "faces" folder

`os.listdir()` returns a list like: `["545895.jpg", "557955.png", "123456.jpeg"]`

```
python
```

```
if filename.endswith((".jpg", ".png", ".jpeg")):
```

Check if file is an image

`.endswith()` checks if a string ends with specific text:

```
python
```

```
"photo.jpg".endswith(".jpg") # True  
"document.pdf".endswith(".jpg") # False
```

The tuple `(".jpg", ".png", ".jpeg")` means "any of these"

```
python
```

```
cms_id = os.path.splitext(filename)[0]
```

Extract student ID from filename

`os.path.splitext()` splits filename into name and extension:

```
python

os.path.splitext("545895.jpg") # Returns ("545895", ".jpg")
```

`[0]` takes just the first part (the ID)

```
python

image_path = os.path.join(folder_path, filename)
```

Create full path to image file

Combines folder and filename properly:

```
python

os.path.join("faces", "545895.jpg") # Returns "faces/545895.jpg"
```

```
python

try:
```

Start a try-except block

What's try-except? Safety net for your code. If something goes wrong inside `try`, jump to `except` instead of crashing.

```
python

try:
    result = 10 / 0 # This will cause an error!
except:
    print("Oops, can't divide by zero!")
```

```
python
```

```
bgr_image = cv2.imread(image_path)
```

Read the image file into memory

`cv2.imread()` loads the image in **BGR format** (Blue-Green-Red, OpenCV's way of storing colors)

python

```
if bgr_image is None:
    print(f"Could not read image: {filename}")
    continue
```

Check if image loaded successfully

- If `imread()` fails, it returns `None`
- `continue` = Skip to next file in the loop

What's an f-string? Way to put variables inside text:

python

```
name = "Ali"
print(f"Hello {name}!") # Prints: Hello Ali!
```

python

```
rgb_image = cv2.cvtColor(bgr_image, cv2.COLOR_BGR2RGB)
```

Convert BGR to RGB

- OpenCV uses BGR (weird, I know!)
 - `face_recognition` library needs RGB
 - `cv2.cvtColor()` converts between color formats
-

python

```
encodings = face_recognition.face_encodings(rgb_image)
```

Generate face encoding

What's a face encoding? A list of 128 numbers that mathematically represent a face. Like a fingerprint for faces!

This function:

1. Detects faces in the image
2. Converts each face to 128 numbers
3. Returns a list (could be multiple faces, or empty if none found)

```
python

print(encodings)
```

Print the encoding to console (for debugging)

```
python

if encodings:
```

Check if any faces were found

In Python, an empty list `[]` is `False`, a list with items is `True`

```
python

face_db[cms_id] = encodings[0]
```

Store face in dictionary

- `face_db[cms_id]` = Use student ID as key
- `encodings[0]` = Take first face found
- Stores like: `{"545895": [0.123, -0.456, ...]}`

```
python

print(f"Loaded face for CMS ID: {cms_id}")
```

Print success message

python

```
else:  
    print(f'No face found in image: {filename}')
```

If no face detected in image, print warning

python

```
except Exception as e:  
    print(f'Error processing {filename}: {e}')
```

Catch any errors

- `Exception as e` = Store the error in variable `e`
 - Print what went wrong instead of crashing
-

python

```
return face_db
```

Return the complete dictionary of all face encodings



Initialize the System

python

```
face_database = load_face_database("faces")
```

Call the function to load all face images from "faces" folder and store in `face_database`

python

```
cap = cv2.VideoCapture(0)
```

Open the camera

- `VideoCapture(0)` = Open default camera (0 is first camera)
- `cap` = Short for "capture", represents camera connection

```
python
```

```
valid_cms_ids = load_cms_ids("students.xlsx")
```

Load list of valid IDs from Excel file

🔧 State Management Variables

```
python
```

```
waiting_for_face = False
scanned_id = None
verification_start_time = None
VERIFICATION_TIMEOUT = 10
```

What are these? Variables that track the program's current state

Variable	Purpose
<code>waiting_for_face</code>	Are we in barcode mode (False) or face mode (True)?
<code>scanned_id</code>	Which ID was just scanned?
<code>verification_start_time</code>	When did face verification start?
<code>VERIFICATION_TIMEOUT</code>	How many seconds to wait for face (10)

Note: `VERIFICATION_TIMEOUT` is in ALL CAPS = constant (shouldn't change)

🔄 Main Program Loop

```
python
```

```
while True:
```

Infinite loop - runs forever until you press 'q'

What's a while loop? Repeats as long as condition is `True`

```
python
```

```
while True: # Always true, so loops forever
    # Your code here
```

```
python
```

```
ret, frame = cap.read()
```

Read one frame from camera

What's happening? `cap.read()` returns TWO values:

1. `ret` = Boolean (True if successful, False if failed)
2. `frame` = The actual image (array of pixels)

This is called "tuple unpacking" - getting multiple values at once:

```
python
```

```
x, y = (10, 20) # x=10, y=20
```

Stage 1: Barcode Scanning

```
python
```

```
if not waiting_for_face:
```

Only run this if we're in barcode scanning mode

- `not waiting_for_face` = If `waiting_for_face` is `False`
- This whole block runs only when looking for barcodes

```
python
```

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Convert to black & white (grayscale)

Why? Barcode detection works better on grayscale:

- Faster (1 color channel instead of 3)
- More accurate (color doesn't matter for barcodes)

```
python

barcodes = decode(gray)
```

Detect barcodes in the image

Returns a **list** of barcode objects found (empty list if none)

```
python

for barcode in barcodes:
```

Loop through each detected barcode (usually just one)

```
python

barcode_data = barcode.data.decode("utf-8")
```

Extract barcode content

- `barcode.data` = Raw bytes (like: `b'545895'`)
- `.decode("utf-8")` = Convert bytes to readable string (`"545895"`)

What's UTF-8? Text encoding standard (how computers store letters)

```
python

print("Scanned ID:", barcode_data)
```

Print to console what was scanned

```
python
```

```
if barcode_data in valid_cms_ids:
```

Check if ID is valid

`in` operator checks if item exists in list:

```
python
```

```
"apple" in ["apple", "banana"] # True  
"grape" in ["apple", "banana"] # False
```

✅ If Valid Barcode:

```
python
```

```
scanned_id = barcode_data
```

Save the ID for face verification stage

```
python
```

```
waiting_for_face = True
```

Switch to Stage 2 (face verification mode)

```
python
```

```
verification_start_time = time.time()
```

Record current time

`time.time()` returns seconds since January 1, 1970 (Unix timestamp)

Example: `1704067200.5` (we use it to calculate elapsed time)

```
python
```

```
print(f'Barcode accepted! Please show your face for verification...')
```

Print instruction

```
python
```

```
x, y, w, h = barcode.rect
```

Get barcode location on screen

Tuple unpacking again! 4 values at once:

- `x` = Horizontal position from left
- `y` = Vertical position from top
- `w` = Width of barcode
- `h` = Height of barcode

```
python
```

```
cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

Draw green rectangle around barcode

Parameter	What It Is
<code>frame</code>	Image to draw on
<code>(x, y)</code>	Top-left corner
<code>(x + w, y + h)</code>	Bottom-right corner
<code>(0, 255, 0)</code>	Color in BGR (Green)
<code>2</code>	Line thickness in pixels

Color in BGR:

- `(0, 255, 0)` = Green
 - `(0, 0, 255)` = Red
 - `(255, 255, 255)` = White
 - `(0, 0, 0)` = Black
-

```
python
```

```
cv2.putText(frame, "BARCODE OK - SHOW FACE", (x, y - 10),  
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
```

Draw text on screen

Parameter	What It Is
<code>frame</code>	Image to draw on
<code>"BARCODE OK - SHOW FACE"</code>	Text to display
<code>(x, y - 10)</code>	Position (10 pixels above barcode)
<code>cv2.FONT_HERSHEY_SIMPLEX</code>	Font style
<code>0.7</code>	Font size (scale)
<code>(0, 255, 0)</code>	Green color
<code>2</code>	Thickness

✗ If Invalid Barcode:

```
python
```

```
else:  
    x, y, w, h = barcode.rect  
    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 0, 255), 2)  
    cv2.putText(frame, "ACCESS DENIED!!!", (x, y - 10),  
                cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 2)
```

Draw RED rectangle and "ACCESS DENIED" text

Same as above, but with `(0, 0, 255)` = Red color

👤 Stage 2: Face Verification

```
python
```

```
else:
```

If `waiting_for_face` is `True`, run this block

python

```
elapsed_time = time.time() - verification_start_time
```

Calculate time passed

Current time minus start time = seconds elapsed

Example:

- Start: `1704067200`
 - Now: `1704067203`
 - Elapsed: `3` seconds
-

python

```
remaining_time = int(VERIFICATION_TIMEOUT - elapsed_time)
```

Calculate countdown

- `int()` = Convert to whole number
 - If 3 seconds passed out of 10: remaining = 7
-

python

```
if elapsed_time > VERIFICATION_TIMEOUT:
```

Check if time ran out (more than 10 seconds)

python

```
print("Verification timeout - please scan again")
waiting_for_face = False
scanned_id = None
continue
```

Timeout handling:

- Print message
 - Reset to Stage 1
 - Clear scanned ID
 - `continue` = Skip rest of loop, start over
-

python

```
cv2.putText(frame, f"Verifying face for ID: {scanned_id}", (10, 30),  
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2)
```

Display which ID we're verifying

Position `(10, 30)` = near top-left corner

python

```
cv2.putText(frame, f"Time remaining: {remaining_time}s", (10, 60),  
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2)
```

Display countdown timer (30 pixels below previous text)

python

```
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

Convert current frame to RGB (face_recognition needs RGB)

python

```
live_faces = face_recognition.face_encodings(rgb_frame)
```

Detect faces in current camera frame

Returns list of face encodings (empty if no faces)

```
python
```

```
if live_faces:
```

If at least one face detected

```
python
```

```
live_encoding = live_faces[0]
```

Get first detected face (assume only one person)

`[0]` = first item in list

```
python
```

```
stored_encoding = face_database.get(scanned_id)
```

Get stored face for this student ID

`.get()` = Dictionary method that safely retrieves value

Returns `None` if key doesn't exist

```
python
```

```
if stored_encoding is not None:
```

Check if we have a stored face for this ID

```
python
```

```
match = face_recognition.compare_faces([stored_encoding], live_encoding)[0]
```

Compare faces

- `compare_faces()` = Compares faces
- `[stored_encoding]` = List of stored faces (must be a list)
- `live_encoding` = Live face from camera
- Returns: `[True]` or `[False]`

- `[0]` = Get first result from list
-

✓ If Faces Match:

python

```
if match:
    result = "✓ ACCESS GRANTED"
    color = (0, 255, 0)
    print(result)
```

Set success message and green color

python

```
cv2.putText(frame, result, (10, 120),
            cv2.FONT_HERSHEY_SIMPLEX, 1.2, color, 3)
```

Display large success message

Font size `1.2` and thickness `3` = big and bold!

python

```
cv2.imshow("Barcode Scanner", frame)
```

Update window with current frame

python

```
cv2.waitKey(3000)
```

Pause for 3 seconds

`cv2.waitKey()` waits for:

- A key press, OR
- The specified milliseconds (3000 = 3 seconds)

This shows the success message long enough to read

python

```
waiting_for_face = False
scanned_id = None
```

Reset system back to Stage 1 for next person

❌ If Faces Don't Match:

python

```
else:
    result = "❌ FACE MISMATCH"
    color = (0, 0, 255)
    print(result)
    cv2.putText(frame, result, (10, 120),
                cv2.FONT_HERSHEY_SIMPLEX, 1.2, color, 3)
    cv2.imshow("Barcode Scanner", frame)
    cv2.waitKey(3000)

    waiting_for_face = False
    scanned_id = None
```

Show RED "FACE MISMATCH", pause, then reset

⚠️ If No Face Registered:

python

```
else:
    result = "❌ FACE NOT REGISTERED"
    color = (0, 0, 255)
    cv2.putText(frame, result, (10, 120),
                cv2.FONT_HERSHEY_SIMPLEX, 1.0, color, 2)
```

Display error - this ID has no face photo in database

⌚ If No Face Detected Yet:

python

else:

```
cv2.putText(frame, " ⌚ Waiting for face...", (10, 120),  
cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)
```

Show waiting message - person hasn't shown face yet

Display and Exit

python

```
cv2.imshow("Barcode Scanner", frame)
```

Show camera window

Creates/updates window with current frame (including all drawings)

python

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

Check if 'q' was pressed

- `cv2.waitKey(1)` = Wait 1 millisecond for key press
- `& 0xFF` = Bitwise operation (handles different systems)
- `ord('q')` = Get numeric code for letter 'q' (113)
- If they match → condition is `True`

What's ord()? Converts character to number:

python

```
ord('a') # Returns 97  
ord('q') # Returns 113
```

python

```
break
```

Exit the loop if 'q' was pressed

`break` = Immediately exit the current loop

```
python  
  
cap.release()
```

Release the camera

Frees up camera so other programs can use it

```
python  
  
cv2.destroyAllWindows()
```

Close all OpenCV windows

Key Programming Concepts

1 Variables

Store information

```
python  
  
name = "Ali"  
age = 20
```

2 Lists

Collection of items

```
python  
  
students = ["545895", "557955", "123456"]
```

3 Dictionaries

Key-value pairs

```
python  
  
grades = {"Ali": 95, "Sara": 88}
```

4 Functions

Reusable code blocks

```
python

def greet(name):
    return f'Hello {name}!'
```

5 Loops

Repeat code

```
python

for i in range(5):
    print(i) # Prints 0,1,2,3,4
```

6 If Statements

Make decisions

```
python

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

7 Try-Except

Handle errors

```
python

try:
    result = 10 / 0
except:
    print("Error!")
```

System Flow Summary

1. **Load Data** → Read valid IDs from Excel, load face images
2. **Start Camera** → Open webcam

3. **Stage 1 Loop** → Continuously scan for barcodes
- Valid barcode → Switch to Stage 2
 - Invalid → Show "ACCESS DENIED"
4. **Stage 2** → Wait 10 seconds for face
- Face matches → "ACCESS GRANTED", reset
 - Face mismatch → "FACE MISMATCH", reset
 - Timeout → Reset to Stage 1
5. **Repeat** until user presses 'q'
-

Color Reference (BGR Format)

Color	BGR Code
 Green	(0, 255, 0)
 Red	(0, 0, 255)
 Blue	(255, 0, 0)
 Yellow	(0, 255, 255)
 White	(255, 255, 255)
 Black	(0, 0, 0)

Important Python Syntax

F-Strings (Formatted Strings)

```
python
name = "Ali"
age = 20
print(f'My name is {name} and I'm {age}')
# Output: My name is Ali and I'm 20
```

Tuple Unpacking

```
python
```

```
x, y = (10, 20) # x=10, y=20
ret, frame = cap.read() # Get 2 values at once
```

List Methods

```
python

my_list = []
my_list.append("item") # Add to end
```

Dictionary Methods

```
python

my_dict = {"key": "value"}
result = my_dict.get("key") # Safe retrieval
```

String Methods

```
python

text = " hello "
text.strip() # Remove spaces: "hello"
text.endswith(".jpg") # Check ending
```

✓ What You Learned

- ✓ **File handling** (Excel, images, folders)
 - ✓ **Computer vision** (camera, barcode, face detection)
 - ✓ **Functions** (reusable code)
 - ✓ **Loops** (for, while)
 - ✓ **Conditionals** (if/else)
 - ✓ **Data structures** (lists, dictionaries)
 - ✓ **Error handling** (try-except)
 - ✓ **State management** (tracking program stages)
-

Next Steps

1. Add entry/exit logging
2. Store access history in database
3. Add numpy for data analysis
4. Create admin dashboard
5. Deploy on actual NUST gates

Great job building this! 